

TechCheck

Documentación Técnica Completa

Sistema de Checklists de Mantenimiento

Versión 1.0.0 · Junio 2026

Stack Tecnológico

Frontend: Angular 19 + Tailwind CSS

Backend: Node.js + Express

Almacenamiento: JSON local

Identificadores: UUID v4

Características Principales

Gestión de proyectos y equipos

Checklists con evidencia fotográfica

Exportar/Importar proyectos

Almacenamiento por proyecto

Tabla de Contenidos

- 1. Descripción General
- 2. Arquitectura del Sistema
- 3. Estructura de Directorios
- 4. Backend — Node.js / Express
 - 4.1 Configuración (.env)
 - 4.2 Punto de Entrada (index.js)
 - 4.3 Capa de Acceso a Datos (dataAccess.js)
 - 4.4 Rutas API
 - 4.5 Almacenamiento de Datos
- 5. Frontend — Angular 19
 - 5.1 Módulos y Componentes
 - 5.2 Modelos de Datos (TypeScript)
 - 5.3 Servicios
 - 5.4 Rutas del Frontend
- 6. Funcionalidades del Sistema
 - 6.1 Gestión de Proyectos
 - 6.2 Gestión de Equipos
 - 6.3 Sistema de Revisiones
 - 6.4 Plantillas de Checklist
 - 6.5 Gestión de Técnicos
 - 6.6 Historial de Revisiones
 - 6.7 Exportar e Importar Proyectos
- 7. API REST — Referencia Completa
- 8. Flujo de Datos
- 9. Instalación y Despliegue
- 10. Variables de Entorno
- 11. Guía de Uso
- 12. Hoja de Ruta (Roadmap)

1. Descripción General

TechCheck es una aplicación web de gestión de checklists de mantenimiento técnico desarrollada para uso interno en entornos empresariales, especialmente orientada a equipos de soporte de TI en la Zona Franca de Rionegro, Antioquia, Colombia.

Permite a los equipos técnicos:

- Organizar el mantenimiento de equipos por proyectos
- Crear checklists reutilizables mediante plantillas
- Registrar revisiones con evidencia fotográfica y archivos adjuntos
- Hacer seguimiento del estado de mantenimiento (Pendiente, En proceso, Terminado)
- Asignar técnicos responsables a equipos
- Exportar e importar proyectos completos como respaldo

Stack Tecnológico

Capa	Tecnología	Versión
Frontend	Angular	19
Estilos	Tailwind CSS	3.x
Backend	Node.js + Express	4.x
Almacenamiento	JSON local (archivos por proyecto)	—
Identificadores	UUID v4	9.x
Configuración	dotenv	16.x

2. Arquitectura del Sistema

NAVEGADOR WEB

■■■ Angular 19 (SPA – Single Page Application)

■ ■■■ Módulo Proyectos / Equipos

■ ■■■ Módulo Plantillas

■ ■■■ Módulo Técnicos

■ ■■■ Módulo Historial

■

■ HTTP / REST API (puerto configurable via .env)

■

BACKEND – Express.js

■■■ /api/proyectos /api/equipos /api/revisiones

■■■ /api/plantillas /api/tecnicos

■■■ dataAccess.js (capa de abstracción de datos)

■

■ fs (sistema de archivos Node.js)

■

ALMACENAMIENTO JSON

```
■■■ data/global.json          ← proyectos, técnicos, plantillas
■■■ data/proyectos/
    ■■■ {proyectoId}.json     ← equipos[] + revisiones[]
```

Nota arquitectural: El frontend compilado es servido estáticamente por el mismo servidor Express. La capa *dataAccess.js* abstrae completamente el origen de datos, preparada para migración futura a PostgreSQL sin modificar las rutas o servicios.

3. Estructura de Directorios

```
techcheck/
■■■ start.bat                ← Script de inicio Windows
■■■ backend/
■   ■■■ .env                 ← Variables de entorno
■   ■■■ index.js             ← Servidor Express
■   ■■■ package.json
■   ■■■ data/
■       ■■■ global.json      ← Proyectos, técnicos, plantillas
■       ■■■ proyectos/
■           ■■■ {uuid}.json   ← Equipos + revisiones por proyecto
■   ■■■ db/
■       ■■■ dataAccess.js     ← Capa de acceso a datos
■   ■■■ routes/
■       ■■■ proyectos.js
■       ■■■ equipos.js
■       ■■■ plantillas.js
■       ■■■ tecnicos.js
■       ■■■ revisiones.js
■■■ frontend/
    ■■■ angular.json
    ■■■ package.json
    ■■■ tailwind.config.js
    ■■■ src/
        ■■■ index.html        ← Incluye XLSX CDN
        ■■■ environments/
        ■■■ app/
            ■■■ app.ts         ← Layout + sidebar
            ■■■ core/
                ■■■ models/models.ts ← Interfaces TypeScript
                ■■■ services/    ← Servicios HTTP
            ■■■ shared/done.pipe.ts ← Pipe para contar ítems
            ■■■ modules/
                ■■■ equipos/
```

```

    ■■■ plantillas/
    ■■■ tecnicos/
    ■■■ historial/
    ■■■ revisiones/

```

4. Backend — Node.js / Express

4.1 Configuración (.env)

```
# ■■■ SERVIDOR ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  
  
PORT=3000  
  
# ■■■ ALMACENAMIENTO ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  
  
# json = archivos locales (actual)  
# postgres = PostgreSQL (futuro)  
  
DATA_SOURCE=json  
  
# ■■■ BASE DE DATOS (para migración futura) ■■■■■  
  
DB_HOST=localhost  
  
DB_PORT=5432  
  
DB_NAME=techcheck  
  
DB_USER=postgres  
  
DB_PASSWORD=  
  
# ■■■ SEGURIDAD (para autenticación futura) ■■■■■  
  
JWT_SECRET=techcheck_secret_key  
  
JWT_EXPIRES_IN=8h  
  
# ■■■ APLICACIÓN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  
  
APP_NAME=TechCheck  
  
APP_VERSION=1.0.0  
  
NODE_ENV=development
```

4.2 Punto de Entrada (index.js)

- Carga variables de entorno con **dotenv**
- Configura CORS y límite de **50MB** para JSON (necesario por imágenes en base64)
- Registra todas las rutas bajo el prefijo **/api/**
- Sirve el frontend compilado de Angular estáticamente
- Redirige rutas no-API al **index.html** para el enrutamiento SPA
- Muestra información del entorno en consola al iniciar

4.3 Capa de Acceso a Datos (dataAccess.js)

Esta capa es el núcleo del almacenamiento. Abstrae completamente el acceso a datos del resto de la aplicación. Cuando se migre a PostgreSQL, solo este archivo cambia.

Función	Descripción
getProyectos()	Lista todos los proyectos desde global.json
createProyecto(obj)	Crea proyecto + archivo JSON vacío en data/proyectos/
deleteProyecto(id)	Elimina proyecto y su archivo JSON
getEquipos()	Lista equipos de todos los proyectos (recorre archivos)
getEquiposByProyecto(id)	Equipos de un proyecto específico
createEquipo(equipo)	Agrega equipo al JSON del proyecto correspondiente
getRevisiones(filtros)	Lista revisiones con filtros opcionales
createRevision(rev)	Guarda revisión en el JSON del proyecto del equipo
exportarProyecto(id)	Genera objeto completo: proyecto+equipos+revisiones+técnicos
importarProyecto(datos)	Importa un proyecto generando nuevos IDs únicos

4.4 Rutas API

Proyectos — /api/proyectos

Método	Ruta	Descripción
GET	/	Lista proyectos con total de equipos
GET	/:id	Obtiene un proyecto por ID
GET	/:id/equipos?estado=	Lista equipos con filtro de estado
GET	/:id/todos-equipos	Lista todos los equipos sin filtrar
GET	/:id/exportar	Exporta el proyecto completo como JSON
POST	/	Crea un nuevo proyecto
POST	/importar	Importa un proyecto desde JSON
PUT	/:id	Actualiza un proyecto
DELETE	/:id	Elimina un proyecto y su archivo de datos

Filtros de estado para /:id/equipos?estado=

Valor	Condición
pendiente	Sin revisión o sin ningún ítem marcado
en_proceso	Al menos 1 ítem marcado, no todos
terminado	Todos los ítems marcados al 100%

Revisiones — /api/revisiones

Método	Ruta	Descripción
GET	/	Lista revisiones (filtros: equipoid, tecnicoid, estado)

Método	Ruta	Descripción
GET	/:id	Obtiene revisión con nombre de equipo y técnico
POST	/	Crea una nueva revisión
PUT	/:id	Actualiza una revisión
DELETE	/:id	Elimina una revisión

4.5 Almacenamiento de Datos — Estructura JSON

Estructura del equipo:

```
{
  "id": "uuid-v4",
  "nombre": "PC-Recepcion",
  "descripcion": "Dell OptiPlex 7090",
  "items": [
    {
      "label": "Verificar antivirus",
      "observacionGuia": "Abrir Avast y verificar ultima actualizacion",
      "archivosGuia": ["data:image/png;base64,..."]
    }
  ],
  "plantillaId": "uuid | null",
  "proyectoIds": ["uuid-proyecto"],
  "tecnicoAsignadoId": "uuid-tecnico | null",
  "tiempoLimite": 3,
  "unidadTiempo": "horas | dias",
  "iniciadoEn": "2026-06-10T10:00:00.000Z",
  "terminadoEn": null,
  "creadoEn": "2026-06-01T00:00:00.000Z",
  "actualizadoEn": "2026-06-01T00:00:00.000Z"
}
```

Estructura de la revisión:

```
{
  "id": "uuid-v4",
  "equipoId": "uuid-equipo",
  "tecnicoId": "uuid-tecnico | null",
  "tecnicoNombre": "Juan Perez",
  "estado": "ok | observacion | problema",
  "items": [
    {
      "label": "Verificar antivirus",
      "checked": true,
      "nota": "Actualizado a version 22.x",
      "archivos": ["data:image/png;base64,..."],
    }
  ]
}
```

```
    "observacionGuia": "...",
    "archivosGuia": []
  }
],
"observacionGeneral": "Todo en orden",
"fotos": ["data:image/png;base64,..."],
"creadoEn": "2026-06-10T10:30:00.000Z",
"actualizadoEn": "2026-06-10T10:30:00.000Z"
}
```


5. Frontend — Angular 19

5.1 Módulos y Componentes

Componente	Ruta	Responsabilidad
app.ts	/	Layout general, sidebar de navegación
equipos-list	/equipos	Proyectos, equipos, revisiones (vista principal)
plantillas-list	/plantillas	CRUD plantillas + importación Excel
tecnicos-list	/tecnicos	CRUD técnicos
historial-list	/historial	Historial de revisiones completadas
revisiones-form	/revisiones	Formulario de revisión independiente

Módulo equipos-list — el más complejo:

Maneja dos vistas internas:

- **Vista 1 — Proyectos:** Cards con total de equipos, botones editar/exportar/eliminar
- **Vista 2 — Equipos del proyecto:** Tres filtros (Pendientes/En proceso/Terminados), cards con estado y progreso
- **Modal de equipo:** Crear/editar con ítems, observaciones guía, archivos guía, técnico y tiempo límite
- **Modal de revisión:** Checklist interactivo con evidencia fotográfica y archivos por ítem

5.2 Modelos de Datos (TypeScript)

```
// Ítem de un equipo (información permanente de guía)
interface ItemEquipo {
  label: string;
  observacionGuia: string;    // Instrucciones para el técnico
  archivosGuia: string[];    // Archivos de referencia (base64)
}

// Ítem dentro de una revisión
interface ItemRevision {
  label: string;
  checked: boolean;
  nota: string;               // Observación del técnico al revisar
  archivos: string[];         // Evidencia fotográfica del ítem (base64)
  observacionGuia?: string;   // Hereda del ItemEquipo (solo lectura)
  archivosGuia?: string[];    // Hereda del ItemEquipo (solo lectura)
}

// Estados posibles de una revisión
type EstadoRevision = 'ok' | 'observacion' | 'problema';
```

5.3 Servicios

Servicio	Archivo	Métodos principales
ProyectosService	proyectos.service.ts	getAll, getById, getEquiposFiltrados, exportarProyecto, importarProyecto
EquiposService	equipos.service.ts	getAll, getById, create, update, delete
PlantillasService	plantillas.service.ts	getAll, create, update, delete
TecnicosService	otros.services.ts	getAll, create, update, delete
RevisionesService	otros.services.ts	getAll, create, delete

5.4 Rutas del Frontend

```
const routes = [
  { path: '',          redirectTo: 'equipos', pathMatch: 'full' },
  { path: 'equipos',   loadChildren: () => import('./modules/equipos/equipos.routes') },
  { path: 'historial', loadChildren: () => import('./modules/historial/historial.routes') },
  { path: 'plantillas', loadChildren: () => import('./modules/plantillas/plantillas.routes') },
  { path: 'tecnicos',  loadChildren: () => import('./modules/tecnicos/tecnicos.routes') },
];
```

Todas las rutas usan **lazy loading** para optimizar el tiempo de carga inicial.

6. Funcionalidades del Sistema

6.1 Gestión de Proyectos

Los proyectos son el contenedor principal. Cada proyecto tiene su propio archivo JSON independiente en `data/proyectos/{id}.json`, lo que permite escalabilidad y copias de seguridad individuales.

Operación	Descripción
Crear proyecto	Nombre requerido + descripción opcional. Crea archivo JSON vacío.
Editar proyecto	Modifica nombre y descripción.
Eliminar proyecto	Elimina el proyecto y su archivo JSON con todos los datos.
Exportar proyecto	Descarga JSON con proyecto, equipos, revisiones y técnicos.
Importar proyecto	Restaura un proyecto exportado con nuevos IDs únicos.

6.2 Gestión de Equipos

Los equipos representan los activos físicos o lógicos a mantener (PCs, servidores, impresoras, switches, etc.)

Campos del equipo:

Campo	Tipo	Descripción
nombre	string (requerido)	Identificador del equipo
descripcion	string (opcional)	Detalles del hardware/software
tecnicoAsignadoId	uuid null	Técnico responsable del equipo
plantillaId	uuid null	Plantilla de checklist aplicada
tiempoLimite	number (opcional)	Tiempo máximo para completar el checklist
unidadTiempo	"horas" "dias"	Unidad del tiempo límite
items	ItemEquipo[]	Lista de tareas del checklist con guías

Ítems del checklist — información de guía permanente:

- **observacionGuia:** Instrucciones para el técnico sobre cómo realizar la tarea
- **archivosGuia:** Imágenes o documentos de referencia (manuales, capturas)
- Esta información es de solo lectura durante la revisión y sirve como guía permanente
- Es diferente de las observaciones de la revisión, que son registros del técnico al revisar

Sistema de filtros automáticos:

Filtro	Condición	Descripción
Pendientes	0 ítems marcados o sin revisión	Equipos que no han sido revisados
En proceso	1+ ítems marcados, no todos	Revisión iniciada pero incompleta
Terminados	Todos los ítems marcados	Revisión completada al 100%

6.3 Sistema de Revisiones

Una revisión es el registro de una sesión de mantenimiento sobre un equipo. El sistema permite guardar revisiones incompletas y retomarlas posteriormente.

Flujo de revisión:

1. El técnico hace clic en Revisar en la tarjeta del equipo
2. Se abre el modal de revisión con el checklist completo
3. Por cada ítem: marcar, ver guía, agregar observación y adjuntar evidencia
4. Definir estado general: OK / Con observaciones / Con problemas
5. Agregar observación general y fotos de evidencia (opcional)
6. Guardar la revisión

Retomar revisión incompleta: Si una revisión se guarda sin completar todos los ítems, al hacer clic en Revisar nuevamente se carga el estado anterior. Al guardar la revisión retomada, la anterior incompleta se elimina automáticamente.

6.4 Plantillas de Checklist

Las plantillas permiten reutilizar listas de ítems en múltiples equipos. Al aplicar una plantilla a un equipo, los ítems se copian y pueden modificarse libremente sin afectar la plantilla original.

Importación desde Excel — Formato requerido:

nombre	descripcion	items
Mantenimiento PC	Revisión periódica	Revisar antivirus Revisar disco Verificar RAM
Revisión Red	Infraestructura	Verificar switch Probar velocidad

Los ítems se separan con el carácter | dentro de la misma celda.

6.7 Exportar e Importar Proyectos

Acción	Cómo	Resultado
Exportar	Botón  en tarjeta de proyecto	Descarga {nombre}_techcheck.json
Importar	Botón Importar proyecto + seleccionar archivo	Nuevo proyecto con nuevos IDs

Estructura del JSON exportado:

```
{
  "proyecto": { "nombre": "...", "descripcion": "..." },
  "equipos": [ { equipos con items y guías } ],
  "revisiones": [ { revisiones completas e incompletas } ],
  "tecnicos": [ { tecnicos del sistema } ]
}
```

7. API REST — Referencia Completa

Formato de respuesta estándar:

```
{
  "success": true | false,
  "data": { ... } | [ ... ],
  "message": "Mensaje de error (solo en errores)"
}
```

Códigos HTTP utilizados:

Código	Uso
200 OK	Operación exitosa (GET, PUT)
201 Created	Recurso creado exitosamente (POST)
400 Bad Request	Error de validación — datos incorrectos o faltantes
404 Not Found	Recurso no encontrado
500 Internal Server Error	Error interno del servidor

Ejemplo — Crear un equipo:

```
POST /api/equipos
Content-Type: application/json

{
  "nombre": "PC-Recepcion",
  "descripcion": "Dell OptiPlex 7090",
  "items": [
    {
      "label": "Verificar antivirus",
      "observacionGuia": "Abrir Avast y verificar version",
      "archivosGuia": []
    }
  ],
  "proyectoIds": ["uuid-del-proyecto"],
  "tecnicoAsignadoId": "uuid-del-tecnico",
  "tiempoLimite": 2,
  "unidadTiempo": "horas"
}
```

Ejemplo — Crear una revisión:

```
POST /api/revisiones
Content-Type: application/json

{
  "equipoId": "uuid-del-equipo",
```

```

"tecnicoId": "uuid-del-tecnico",
"estado": "ok",
"items": [
  {
    "label": "Verificar antivirus",
    "checked": true,
    "nota": "Actualizado correctamente a v22",
    "archivos": []
  }
],
"observacionGeneral": "Equipo en buen estado",
"fotos": []
}

```

8. Flujo de Datos

Ciclo de vida completo de un mantenimiento:

1. CREAR PROYECTO
 - Nombre + descripción opcional
 - Se crea archivo data/proyectos/{id}.json vacío
2. AGREGAR EQUIPO AL PROYECTO
 - Nombre, items con guías, técnico, tiempo límite
 - Se guarda en data/proyectos/{projectId}.json → equipos[]
3. REALIZAR REVISIÓN
 - Técnico abre modal, marca ítems, agrega notas y archivos
 - Guarda → revisión en data/proyectos/{projectId}.json → revisiones[]
 - Si retoma incompleta → elimina anterior, crea nueva actualizada
4. CLASIFICACIÓN AUTOMÁTICA
 - Pendiente: sin revisión o 0 ítems marcados
 - En proceso: 1+ ítems marcados, no todos
 - Terminado: todos los ítems marcados (100%)
5. HISTORIAL
 - Solo revisiones con 100% de ítems marcados
 - Organizadas y filtradas por proyecto

9. Instalación y Despliegue

Requisitos previos:

Software	Versión mínima
Node.js	18.x o superior
npm	9.x o superior
Sistema operativo	Windows 10/11 (o cualquier OS con Node.js)

Instalación:

```
# 1. Instalar dependencias del backend
cd techcheck/backend
npm install

# 2. Instalar dependencias del frontend
cd ../frontend
npm install

# 3. Compilar el frontend para producción
npx ng build --configuration=production

# 4. Iniciar la aplicación
cd ../backend
node index.js

# Abrir en navegador: http://localhost:3000
```

Inicio rápido en Windows:

Doble clic en **start.bat** para iniciar directamente el servidor.

Solución de problemas de compilación:

```
# Limpiar caché de Angular y recompilar
cd techcheck/frontend
npx ng cache clean
npx ng build --configuration=production
```

10. Variables de Entorno

Variable	Por defecto	Descripción
PORT	3000	Puerto del servidor Express
DATA_SOURCE	json	Fuente de datos (json / postgres)
DB_HOST	localhost	Host de PostgreSQL (futuro)
DB_PORT	5432	Puerto de PostgreSQL (futuro)
DB_NAME	techcheck	Nombre de la base de datos (futuro)

Variable	Por defecto	Descripción
DB_USER	postgres	Usuario de PostgreSQL (futuro)
DB_PASSWORD	(vacío)	Contraseña de PostgreSQL (futuro)
JWT_SECRET	techcheck_secret_key	Clave para tokens JWT (futuro)
JWT_EXPIRES_IN	8h	Expiración de tokens JWT (futuro)
APP_NAME	TechCheck	Nombre de la aplicación
APP_VERSION	1.0.0	Versión actual
NODE_ENV	development	Entorno (development / production)

11. Guía de Uso

Primer uso — Configuración inicial

1. Crear los **técnicos** que usará el sistema (menú Técnicos)
2. Crear **plantillas** de checklist reutilizables (menú Plantillas) o importarlás desde Excel
3. Crear el primer **proyecto** (botón + Nuevo proyecto)
4. Dentro del proyecto, **agregar equipos** con sus ítems, técnico y tiempo límite

Realizar una revisión

1. Entrar al proyecto correspondiente
2. En la pestaña **Pendientes**, localizar el equipo a revisar
3. Clic en **Revisar**
4. Verificar cada ítem: marcar, ver guía, agregar observaciones y evidencia
5. Definir el estado general y agregar observación general si aplica
6. Clic en **Guardar revisión**

Hacer copia de seguridad de un proyecto

1. En la vista de proyectos, pasar el mouse sobre el proyecto
2. Clic en el botón ■■ (exportar)
3. Se descarga el archivo JSON con todos los datos del proyecto

Restaurar un proyecto

1. En la vista de proyectos, clic en **Importar proyecto**
2. Seleccionar el archivo JSON exportado previamente
3. El proyecto se crea con todos sus equipos y revisiones (nuevos IDs)
4. Los técnicos nuevos se agregan al sistema automáticamente si no existen

12. Hoja de Ruta (Roadmap)

Corto plazo

- Temporizador en vivo con alerta visual cuando se excede el tiempo límite
- Indicador de tiempo excedido en las tarjetas de equipos
- Notificaciones en pantalla al superar el tiempo asignado

Mediano plazo

- Migración a PostgreSQL (infraestructura preparada en dataAccess.js y .env)
- Sistema de autenticación con JWT (infraestructura preparada en .env)
- Dashboard con estadísticas de mantenimiento (equipos revisados, tiempos, técnicos)
- Exportar reportes de revisiones en PDF

Largo plazo

- Aplicación móvil (PWA — Progressive Web App)
- Notificaciones por correo electrónico

- Soporte multi-empresa / multi-tenant
- Integración con sistemas ERP (Odoo, SAP)

Documentación generada — TechCheck v1.0.0 — Junio 2026